

Histórico e navegação entre commits

O histórico é a linha do tempo da evolução do projeto. Hoje vamos consultá-lo com `git log`, abrir um commit por dentro, comparar versões e até visitar um ponto do passado — tudo com segurança, sem perder nada.

Cada commit guarda muito mais que arquivos



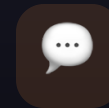
Autor



Data



Horário



Mensagem



Hash único



Cada commit representa **um momento específico** da evolução do projeto.

Visualizando o histórico

bash — git log

```
$ git log
commit a3f7d8e...
Author: Ana Silva <ana@email.com>
Date: Tue Jul 15 10:30:00 2025
Atualiza README com descrição do projeto

commit 9d41b5c...
Author: Ana Silva <ana@email.com>
Cria README inicial
```

O hash é a “impressão digital” única de cada commit.

⚡ GIT LOG --ONELINE

Uma forma rápida de consultar

bash — versão compacta

```
$ git log --oneline
```

```
a3f7d8e Atualiza README com descrição do projeto
```

```
9d41b5c Cria README inicial
```



`git log --oneline` mostra hash reduzido + mensagem: perfeito para consultas rápidas.

 GIT SHOW

O que mudou em um commit específico

 bash — inspecionar um commit

```
$ git show a3f7d8e
```

```
Author: Ana Silva <ana@email.com>
```

```
Atualiza README com descrição do projeto
```

```
+ Este projeto guarda meus estudos de Git.
```

```
- Projeto de estudos.
```



`git show <hash>` abre um commit específico e mostra **exatamente o que mudou nele.**

Comparando versões

bash — o que ainda não comitei

```
$ git diff
```

```
+ ## Contato
```

```
# mostra o que mudou desde o último commit (ainda sem git add)
```

comparar dois commits

```
$ git diff 9d41b5c a3f7d8e
```

```
# mostra a diferença entre dois pontos do histórico
```



git diff responde: o que mudou de uma versão para outra?

 HEAD

“Você está aqui” no histórico



HEAD é o marcador que aponta para o commit onde você está **agora** — normalmente o mais recente.

 VISITAR O PASSADO

Voltar a um commit antigo (e retornar)

 bash — visitar um commit

```
$ git checkout 9d41b5c  
You are in 'detached HEAD' state...  
# não é erro! você só está "visitando" um commit antigo
```

 voltar para o presente

```
$ git switch main  
# HEAD volta para o commit mais recente
```

💡 “Detached HEAD” só significa que o HEAD não está preso a uma branch. Nada foi perdido.

 HISTÓRICO ÚTIL

Você entenderia só lendo a mensagem?

RUINS

teste
alterações
atualização
123



BOAS

Cria arquivo README
Adiciona descrição do projeto
Atualiza título da página
Corrige erro de digitação

✦ BOAS PRÁTICAS

Comece a mensagem com um verbo



Adiciona



Atualiza



Remove



Corrige



Cria



Commits pequenos

Um commit por ideia deixa o histórico organizado.



Commits frequentes

Não espere terminar o projeto inteiro.

 PRÁTICA GUIADA

Fazendo o histórico crescer

✓ Adicionar uma seção nova ao `README.md` .

✓ `git add + git commit -m "..."` com boa mensagem.

✓ Rodar `git log` e `git log --oneline` .

✓ Ler as próprias mensagens: elas fazem sentido?

ENCERRAMENTO

O histórico é comunicação

Cada commit faz parte da história do projeto, e um bom histórico depende de mensagens claras e objetivas.



Você sabia?

Em muitas empresas, o histórico de commits é usado para entender como uma funcionalidade evoluiu, investigar quando um problema surgiu e até gerar automaticamente notas de versão (release notes).



Próxima aula: conhecer o **GitHub** e criar nosso primeiro repositório remoto.